

ICT171 Reflective Journal

MU BRG-ISEA

Introduction to Server Environments and Architectures

Item	Details
Student Name	Khor Woun Hang
Student ID	CT0371957
Additional Server Service	Docker Containerisation
GitHub Repository Link	https://github.com/Selkhor/ICT171-Lab-Journal
Video URL Link / uploaded filename	https://m365kloud.sharepoint.com/sites/LEARN/_layouts/15/stream.aspx?id=%2Fsites%2FLEARN%2FShared%2F0Documents%2F%2DKAPLAN%2FMU%2DISEA%2Fstudent%2FVideoUpload%2FCT0371957%2DKhorWounHang%2DAssignmentISEA%2Emp4&referrer=StreamWebApp%2EWeb&referrerScenario=AddressBarCopied%2Eview%2Edcd9a86b%2D7e8c%2D4520%2Daade%2D691d3b9dab92 CT0371957-KhorWounHang-AssignmentISEA.MP4
GitHub pages saved as PDF	CT0371957-KhorWounHang-AssignmentISEA-GIT01.pdf

Word DOCX file uploaded on LMS	CT0371957-KhorWounHang-AssignmentISEA.docx
--------------------------------	--

Submission Date:17 June 2026

1. Introduction

This reflective journal documents my learning journey throughout ICT171. The unit introduced me to Linux system administration, cloud computing, scripting, automation, DNS management, web security, and containerisation technologies. All practical activities were completed using Ubuntu 22.04 ARM64 running on UTM virtualisation software on a MacBook M1. Cloud-based exercises were completed using Amazon Web Services (AWS) EC2.

Before starting ICT171, I knew very little about Linux and had no real experience with cloud computing. At the beginning, many of the commands and concepts seemed unfamiliar. As the semester progressed, I gradually became more comfortable working with Linux, AWS, Bash scripting, DNS, HTTPS, and Docker. This journal describes the activities I completed, the challenges I faced, the solutions I found, and how these skills relate to real-world IT environments.

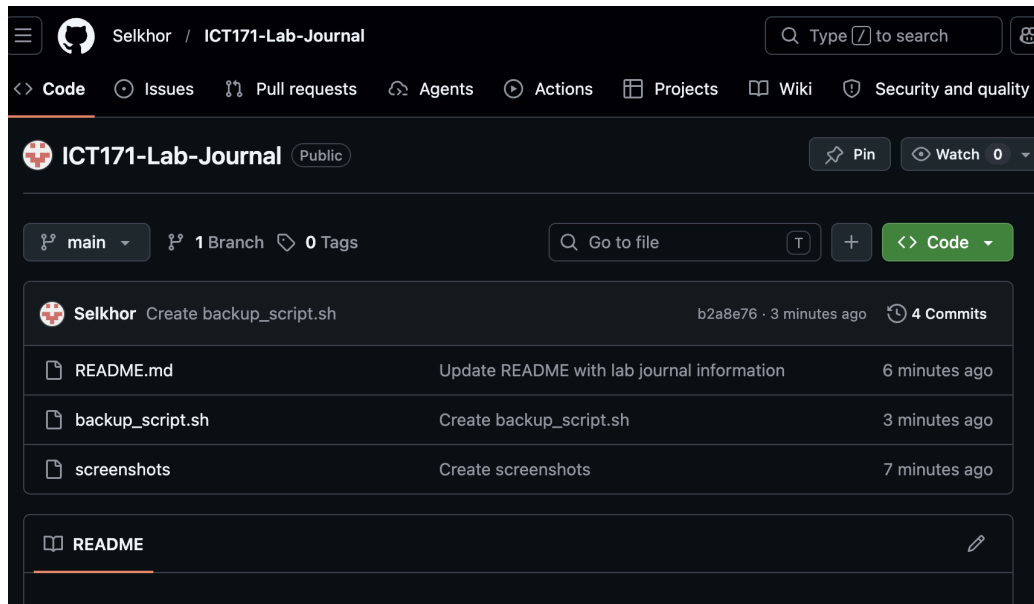
2. Linux Environment Setup

2.1 GitHub Repository

At the beginning of the semester, I created a GitHub repository called ICT171-Lab-Journal to store screenshots, scripts, notes, and laboratory evidence.

Using GitHub allowed me to organise my work systematically and maintain version control throughout the semester. Every major task and script was documented and uploaded to the repository.

Screenshot 1



Reflection: I thought GitHub was just for code. Then I made my own repo for this class. Called it ICT171-Lab-Journal. Threw all my screenshots, scripts and notes in there. After each lab I just uploaded everything. One time I messed up a script. Went back to an old version and fixed it. Saved me. At work I guess teams would do the same with config files.

2.2 Installing Ubuntu on Apple Silicon

Because I use a MacBook M1, traditional virtualisation platforms such as VirtualBox were unsuitable. Instead, I used UTM to create an Ubuntu 22.04 ARM64 virtual machine.

Initially, I downloaded the x86_64 version of Ubuntu, which caused compatibility problems. After identifying the issue, I downloaded the ARM64 version and successfully installed Ubuntu.

Verification was performed using:

```
bash
```

```
uname -m
```

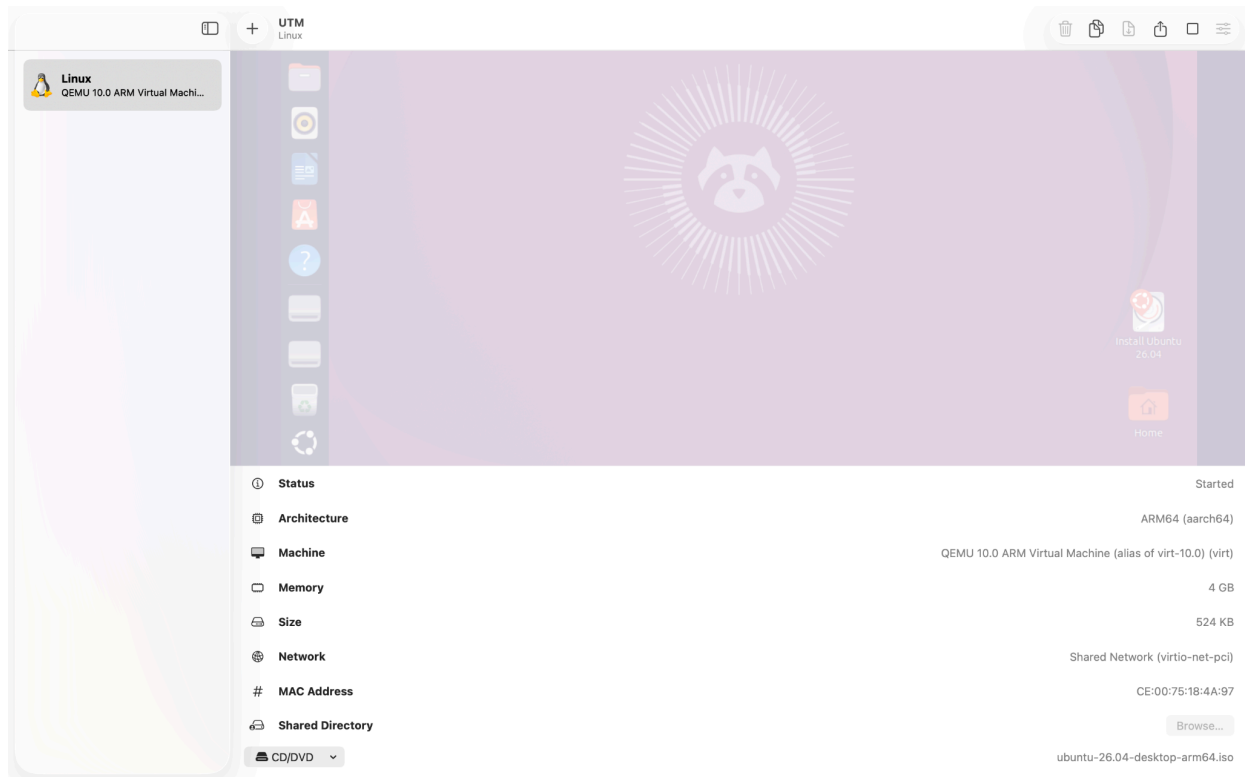
which returned:

```
bash
```

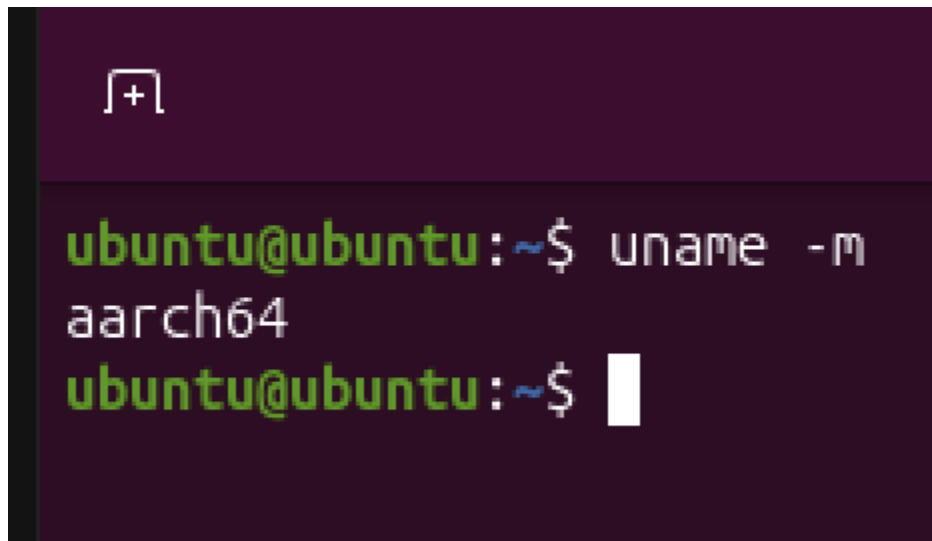
```
aarch64
```

confirming the operating system was running on ARM64 architecture.

Screenshot 2



Screenshot 3



Reflection: I made a mistake at the start: I downloaded the x86_64 version of Ubuntu for my M1 Mac. When I tried to install it, nothing worked. After some searching online, I realised the problem — my Mac uses ARM64 architecture, not x86. That's when I found UTM and downloaded the correct ARM64 ISO. Later in the semester, this experience paid off when I launched AWS Graviton instances. They use the same ARM architecture, so I already understood why they exist and how they're different from regular x86 servers.

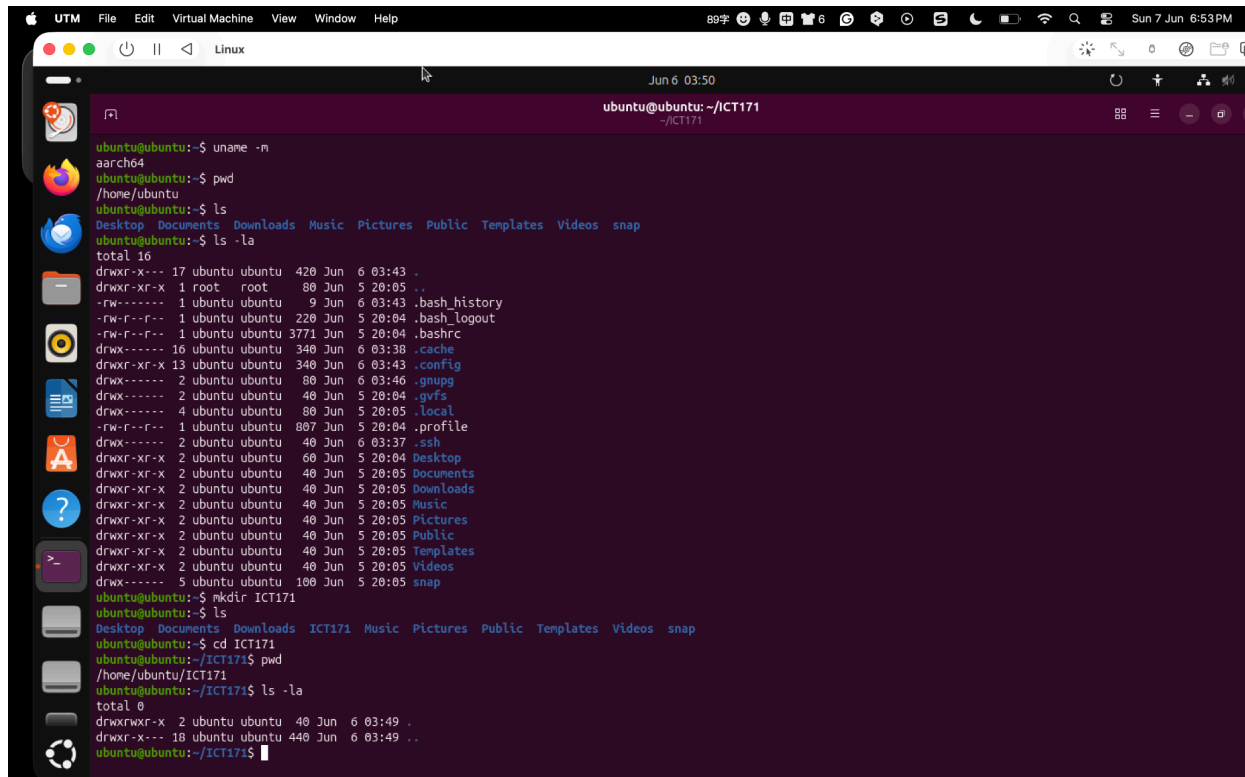
2.3 Ubuntu Command Line Familiarisation

I learned a variety of Linux commands including:

Command	Purpose
ls -la	List all files with permissions
cd, pwd	Navigate directories
mkdir, touch	Create directories and files
cp, mv, rm	Copy, move, delete files
grep, find	Search within files and directories
sudo systemctl	Manage services
ip a, ping	Network configuration and testing

I also installed software using the APT package manager and compiled simple programs using GCC.

Screenshot 4



Reflection: At first, using the command line felt uncomfortable because I was more familiar with graphical user interfaces. However, after repeated practice I realised that command-line tools offer greater efficiency, flexibility, and automation potential. These commands became the foundation for every subsequent activity in the unit.

3. Linux Services, Permissions and File Searching

3.1 Linux Services

Apache was installed and managed using:

```
bash
```

```
sudo apt install apache2 -y
```

```
sudo systemctl start apache2
```

```
sudo systemctl enable apache2
```

Firewall rules were configured using UFW to allow SSH, HTTP, and HTTPS traffic.

```
bash
```

```
sudo ufw allow 22,80,443/tcp
```

```
sudo ufw enable
```

SSH key-based authentication was also tested.

Screenshot 5

```

ubuntu@ubuntu:~/ICT171$ systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-06-07 18:57:02 +08; 1min 26s ago
     Invocation: 20194b69834c47f486e60cb26b151123
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 44363 (apache2)
      Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
        Tasks: 55 (limit: 4106)
       Memory: 18.2M (peak: 18.6M)
          CPU: 224ms
      CGroup: /system.slice/apache2.service
              └─44363 /usr/sbin/apache2 -k start -DFOREGROUND
                 └─44379 /usr/sbin/apache2 -k start -DFOREGROUND
                    └─44380 /usr/sbin/apache2 -k start -DFOREGROUND

Jun 07 18:57:02 ubuntu systemd[1]: Starting apache2.service - The Apache HTTP Server...
Jun 07 18:57:02 ubuntu apachectl[44363]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName'
Jun 07 18:57:02 ubuntu systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-18/18 (END)...skipping...
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-06-07 18:57:02 +08; 1min 26s ago
     Invocation: 20194b69834c47f486e60cb26b151123
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 44363 (apache2)
      Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
        Tasks: 55 (limit: 4106)
       Memory: 18.2M (peak: 18.6M)
          CPU: 224ms
      CGroup: /system.slice/apache2.service
              └─44363 /usr/sbin/apache2 -k start -DFOREGROUND
                 └─44379 /usr/sbin/apache2 -k start -DFOREGROUND
                    └─44380 /usr/sbin/apache2 -k start -DFOREGROUND

Jun 07 18:57:02 ubuntu systemd[1]: Starting apache2.service - The Apache HTTP Server...
Jun 07 18:57:02 ubuntu apachectl[44363]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName'
Jun 07 18:57:02 ubuntu systemd[1]: Started apache2.service - The Apache HTTP Server.
~
~

```

Screenshot 6

```

ubuntu@ubuntu:~/ICT171$ sudo ufw status
Status: active

To Action From
--
22/tcp ALLOW Anywhere
80/tcp ALLOW Anywhere
443/tcp ALLOW Anywhere
22/tcp (v6) ALLOW Anywhere (v6)
80/tcp (v6) ALLOW Anywhere (v6)
443/tcp (v6) ALLOW Anywhere (v6)

```

Reflection: Before this lab, I did not really understand how Linux services worked. I learned how to use systemctl to start, stop, and check Apache. This helped me understand how administrators manage servers and keep services running properly. I also learned that installing software is only one part of the process. Firewall settings and access control are just as important. Overall, this activity helped me understand how Linux servers are managed in real environments.

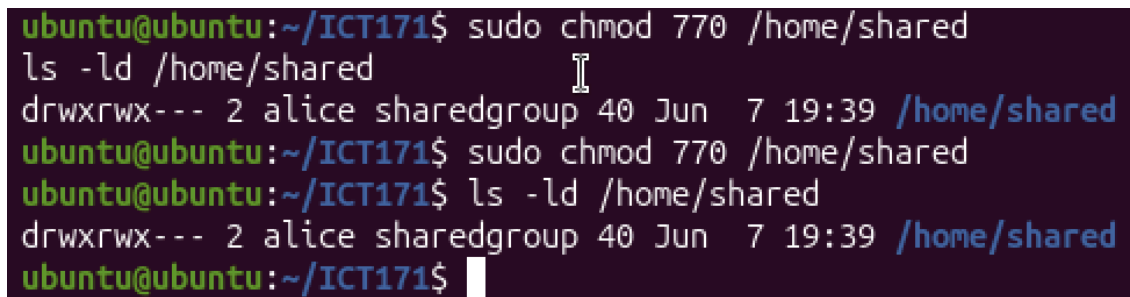
3.2 Linux Permissions

User accounts and groups were created to explore Linux access control mechanisms.

User	Access Required	Command
alice	Read, Write, Execute (rwx)	chmod 770
bob	Read and Execute only (r-x)	chmod 750
mallory	No access (---)	chmod 000

Permissions were modified using chmod, chown, and chgrp.

Screenshot 7



```
ubuntu@ubuntu:~/ICT171$ sudo chmod 770 /home/shared
ls -ld /home/shared
drwxrwx--- 2 alice sharedgroup 40 Jun 7 19:39 /home/shared
ubuntu@ubuntu:~/ICT171$ sudo chmod 770 /home/shared
ubuntu@ubuntu:~/ICT171$ ls -ld /home/shared
drwxrwx--- 2 alice sharedgroup 40 Jun 7 19:39 /home/shared
ubuntu@ubuntu:~/ICT171$
```

Reflection: I'll admit — at first I just used chmod 777 whenever something said 'permission denied'. It worked, so I didn't think about it. But then I created users alice, bob, and mallory and tried to give them different access levels. I saw that 777 gives everyone full access to everything. That's dangerous. Now I use 750 or 770 and only give exactly what each user needs. This 'least privilege' idea makes a lot of sense for keeping systems secure.

3.3 File Searching and Analysis

The find and grep commands were used to search large datasets and locate specific files.

Examples included:

bash

```
find ./Gutenberg -name "*.txt"
```

```
grep -r "verdigris" ./Gutenberg
```

These tools enabled efficient searching of thousands of files. I successfully located answers to lab questions including counting keyword occurrences and identifying authors of specific files.

Screenshot 8


```

ubuntu@ubuntu:~/ICT171$ find . -name "*.txt"
./file3.txt
./file2.txt
./file1.txt
./test2.txt
./test1.txt
./notes.txt
ubuntu@ubuntu:~/ICT171$ grep -r "linux" .
./file3.txt:linux permissions
./file1.txt:hello linux
ubuntu@ubuntu:~/ICT171$

```

Reflection: Before using find and grep, I would normally search for files manually. After working with larger datasets, I realised how much faster command-line tools are. Being able to locate files and specific keywords within seconds showed me why these tools are so useful for system administration and troubleshooting.

4. Cloud Infrastructure and TCO Analysis

4.1 Total Cost of Ownership (TCO)

A Total Cost of Ownership (TCO) comparison was conducted between Amazon Web Services (AWS) EC2 and Microsoft Azure Virtual Machines over a three-year period.

Cost Component	AWS EC2	Azure VM
Virtual Machine Instance	\$102.48	\$118.80
Storage	Included	Included
Public IP Address	Included	Included
Operating System	Ubuntu Linux (Free)	Ubuntu Linux (Free)
SSL Certificate	Free (Let's Encrypt)	Free (Let's Encrypt)
Domain Name (3 Years)	\$19.07	\$19.07
3-Year Total Cost	\$121.55	\$137.87

The comparison indicated that both cloud providers offered similar capabilities, including virtual machines, networking, storage, and security features. AWS EC2 provided a slightly lower estimated cost, while Azure offered strong integration with Microsoft services and enterprise environments.

	A	B	C
1	Cost Component	AWS EC2	Azure VM
2	Virtual Machine Instance (3 Years)	\$102.48	\$118.80
3	Storage	Included	Included
4	Public IP Address	Included	Included
5	Operating System	Ubuntu Linux (Free)	Ubuntu Linux (Free)
6	SSL Certificate	Free (Let's Encrypt)	Free (Let's Encrypt)
7	Domain Name (3 Years)	\$19.07	\$19.07
8	3-Year Total Cost	\$121.55	\$137.87
9			

Reflection: Before this activity, I mainly focused on cost when comparing cloud services. After researching AWS and Azure, I realised that organisations also need to consider security, performance, scalability, and existing technology requirements. This activity helped me understand that choosing a cloud provider involves more than comparing prices and requires balancing both technical and business needs.

4.2 AWS EC2 Deployment

An Ubuntu ARM64 AWS Graviton instance (t4g.micro) was launched and configured. AWS Graviton instances generally provide better price-performance than comparable x86 instances.

SSH access was established using:

```
bash
```

```
chmod 400 aws-key.pem
```

```
ssh -i aws-key.pem ubuntu@server-ip
```

Apache was installed and tested successfully through the public IP address.

Security Groups were configured to allow:

- SSH (22)
- HTTP (80)
- HTTPS (443)

An AWS Budget alert was also configured to monitor costs.

Screenshot 10

Instance summary for i-09a9a5d01c9d5b2be (ICT171-WebServer) [Info](#)

Updated less than a minute ago

Instance ID

 i-09a9a5d01c9d5b2be

Public IPv4 address

 44.195.61.45 | [open address](#) 

IPv6 address

—

Instance state

 Running

Screenshot 11

```
Last login: Thu Jun  4 16:51:41 on console
hansolo@MacBookAir ~ % cd ~/Downloads
ls *.pem
ICT171-Key.pem
hansolo@MacBookAir Downloads % chmod 400 ICT171-Key.pem
hansolo@MacBookAir Downloads % ssh -i ICT171-Key.pem ubuntu@44.195.61.45
The authenticity of host '44.195.61.45 (44.195.61.45)' can't be established.
ED25519 key fingerprint is: SHA256:RJWg0DF17ZIGyfnJWJursbiftEQGKlhtxmlaUfKiJk
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '44.195.61.45' (ED25519) to the list of known hosts.
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1004-aws aarch64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sun Jun  7 13:17:46 UTC 2026

System load:  0.0           Temperature:   -273.1 C
Usage of /:   29.9% of 6.71GB Processes:    149
Memory usage: 36%          Users logged in: 0
Swap usage:   0%           IPv4 address for ens5: 172.31.6.249

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

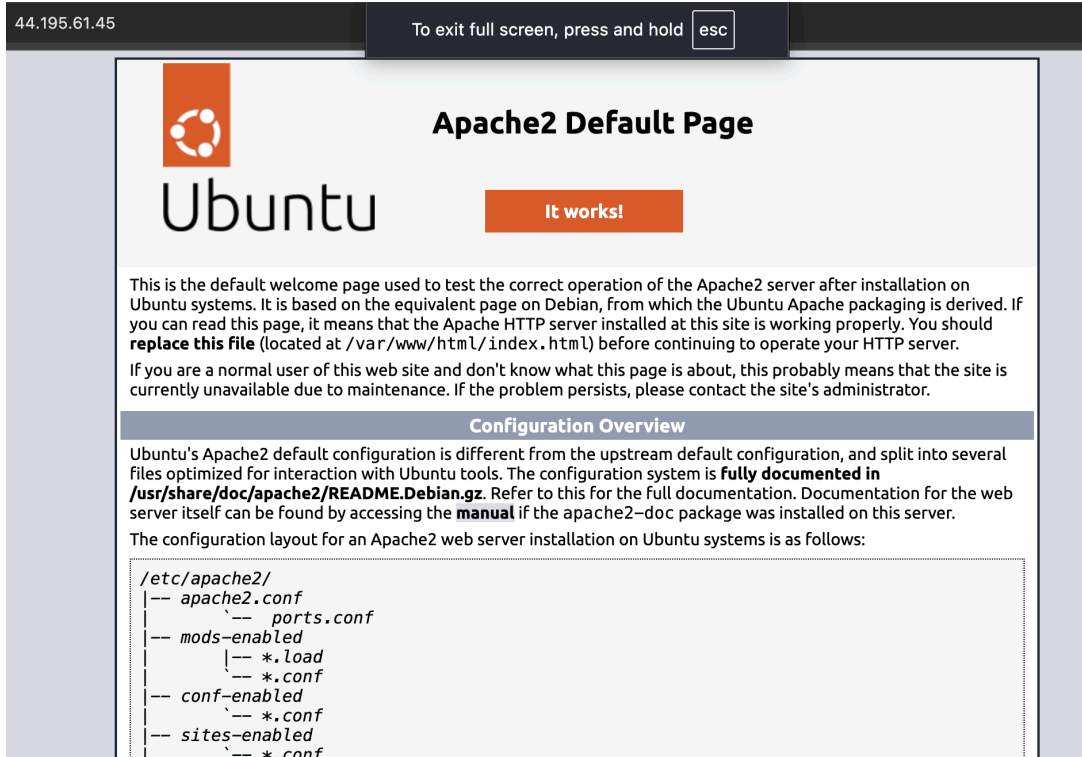
The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

ubuntu@ip-172-31-6-249:~$
```

Screenshot 12



Reflection: Before this lab, cloud computing was mostly something I read about in textbooks. Launching and managing my own EC2 instance made the concept much more practical. It helped me understand how real organisations deploy services on cloud infrastructure while balancing security, accessibility, and cost management.

5. Bash Scripting and Automation

5.1 Bash Scripting

A backup automation script was created to:

- Copy files recursively
- Compress data using zip
- Create timestamped backups
- Upload backups to AWS using SCP

```
#!/bin/bash
```

```
DATE=$(date +%Y%m%d_%H%M%S)
```

```
mkdir -p /home/ubuntu/backup
```

```
cp -R /home/ubuntu/Documents/* /home/ubuntu/backup/
```

```
zip -r "/home/ubuntu/backup_$(date).zip" /home/ubuntu/backup/*
```

```
echo "Backup completed at $(date)"
```

Screenshot 13

```
ubuntu@ip-172-31-6-249:~$ cat /home/ubuntu/backup_script.sh
#!/bin/bash
[DATE=$(date +%Y%m%d_%H%M%S)
mkdir -p /home/ubuntu/backup
cp -R /home/ubuntu/Documents/* /home/ubuntu/backup/
zip -r "/home/ubuntu/backup_$DATE.zip" /home/ubuntu/backup/*
echo "Backup completed at $(date)"
```

Screenshot 14

```
ubuntu@ip-172-31-6-249:~$ bash /home/ubuntu/backup_script.sh
[ adding: home/ubuntu/backup/testfile.txt (stored 0%)
[Backup completed at Wed Jun 10 10:11:12 UTC 2026
```

Reflection: Before this lab, I had never written a script. I just typed commands one by one. When I created my backup script, I realised I could automate repetitive tasks instead of running commands manually every time. My script generated timestamped backup files automatically, which made the process much more efficient. Seeing the script run successfully gave me confidence to learn more about automation. I now understand why scripting is such an important skill for Linux administrators and DevOps engineers.

5.2 Log Analysis and Regular Expressions

Regular expressions and command-line utilities such as `grep` and `awk` were used to analyse Apache logs.

Example:

`bash`

```
grep -E '([0-9]{1,3}\.){3}[0-9]{1,3}' access.log | awk '{print $1, $9}' | sort | uniq -c | sort -nr | head -10
```

This command extracts IP addresses, counts occurrences, and displays the most frequent visitors.

Screenshot 15

```
ubuntu@ip-172-31-6-249:~$ cat > access.log << EOF
192.168.1.10 - - [09/Jun/2026] "GET /index.html" 200
192.168.1.10 - - [09/Jun/2026] "GET /about.html" 200
10.0.0.5 - - [09/Jun/2026] "GET /login" 404
[EOF
ubuntu@ip-172-31-6-249:~$ grep -E '([0-9]{1,3}\.){3}[0-9]{1,3}' access.log
192.168.1.10 - - [09/Jun/2026] "GET /index.html" 200
192.168.1.10 - - [09/Jun/2026] "GET /about.html" 200
10.0.0.5 - - [09/Jun/2026] "GET /login" 404
```

Reflection: Before this activity, I had never looked at server log files in detail. At first, the logs seemed difficult to understand because there was so much information displayed on the screen. After using `grep`, `awk`, and regular expressions, I was able to filter the results and focus only on the information I needed. This helped me understand why system administrators and security teams rely on command-line tools when analysing large amounts of log data.

6. DNS and HTTPS Security

6.1 DNS Configuration

A domain name was linked to the AWS server using DNS A records.

Record Type	Name	Value
A	@	44.195.61.45
A	www	44.195.61.45

Verification was performed using:

bash

nslookup yourdomain.com

Round-robin DNS behaviour was also explored by assigning multiple IP addresses to the same hostname.

Screenshot 16

```
bash-3.2$ nslookup 44.195.61.45
[Server:      192.168.50.1
Address:      192.168.50.1#53

Non-authoritative answer:
45.61.195.44.in-addr.arpa      name = ec2-44-195-61-45.compute-1.amazonaws.com.

Authoritative answers can be found from:
```

Screenshot 17

```
bash-3.2$ dig -x 44.195.61.45
[
; <<>> DiG 9.10.6 <<>> -x 44.195.61.45
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 31468
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:;, udp: 512
;; QUESTION SECTION:
;45.61.195.44.in-addr.arpa.      IN      PTR

;; ANSWER SECTION:
45.61.195.44.in-addr.arpa. 300 IN      PTR      ec2-44-195-61-45.compute-1.amazonaws.com.

;; Query time: 61 msec
;; SERVER: 192.168.50.1#53(192.168.50.1)
;; WHEN: Wed Jun 10 17:55:49 +08 2026
;; MSG SIZE rcvd: 108
```

Reflection: Before this lab, I viewed DNS as a simple name-to-IP translation service. I learned that DNS also supports load balancing, redundancy, and service availability. This activity highlighted the critical role DNS plays in modern network infrastructure.

6.2 HTTPS with Let's Encrypt

HTTPS was implemented using Certbot and Let's Encrypt.

bash

sudo snap install --classic certbot

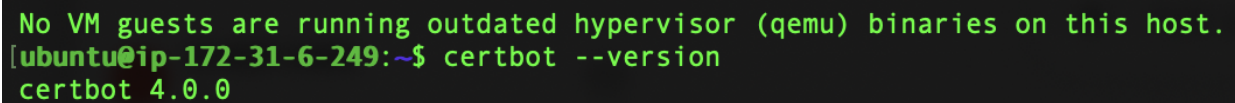
sudo certbot --apache

Certificates were successfully issued and configured for automatic renewal.

bash

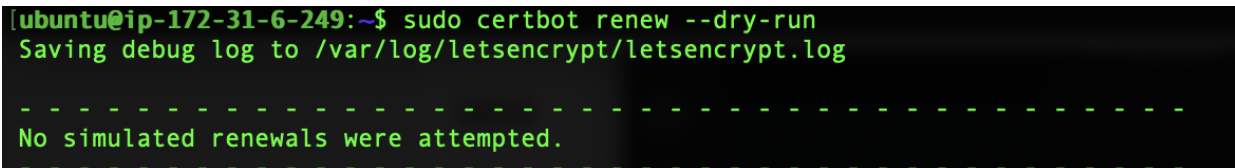
sudo certbot renew --dry-run

Screenshot 18



```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
[ubuntu@ip-172-31-6-249:~]$ certbot --version
certbot 4.0.0
```

Screenshot 19



```
[ubuntu@ip-172-31-6-249:~]$ sudo certbot renew --dry-run
Saving debug log to /var/log/letsencrypt/letsencrypt.log

-----
No simulated renewals were attempted.
-----
```

Reflection: Before working on this activity, I rarely thought about what HTTPS was actually doing behind the scenes. Learning about TLS certificates helped me understand how encrypted communication protects data travelling between users and servers. It also showed me why HTTPS has become a standard security requirement for modern websites.

7. Cron Automation

Cron was used to schedule automated backups.

bash

crontab -e

Added: 0 * * * * /home/ubuntu/backup_script.sh

Backup files were generated automatically each hour.

Screenshot 20


```

ubuntu@ip-172-31-6-249:~$ crontab -e
no crontab for ubuntu - using an empty one
Select an editor. To change later, run select-editor again.
 1. /bin/nano      <---- easiest
 2. /usr/bin/vim.basic
 3. /usr/bin/vim.tiny
 4. /bin/ed

Choose 1-4 [1]: 1
No modification made
ubuntu@ip-172-31-6-249:~$ crontab -l
no crontab for ubuntu
ubuntu@ip-172-31-6-249:~$ crontab -e
no crontab for ubuntu - using an empty one
crontab: installing new crontab
ubuntu@ip-172-31-6-249:~$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
0 2 * * * /home/ubuntu/backup_script.sh >> /home/ubuntu/backup.log 2>&1

```

Screenshot 21

```

[ubuntu@ip-172-31-6-249:~$ nano backup_script.sh
ubuntu@ip-172-31-6-249:~$ chmod +x backup_script.sh
[ubuntu@ip-172-31-6-249:~$ ./backup_script.sh
adding: backup_info.txt (stored 0%)
[ubuntu@ip-172-31-6-249:~$ ls -lh *.zip
-rw-rw-r-- 1 ubuntu ubuntu 214 Jun  9 11:12 backup_20260609_111237.zip

```

Reflection: I set up my cron job to run every hour, but nothing happened. My script worked fine when I ran it manually, so I was confused. After some research, I found out that cron doesn't use the same PATH as my terminal. I had to change all my paths to absolute paths, like /home/ubuntu/backup_script.sh instead of ./backup_script.sh. This was frustrating but taught me an important lesson: always test your automation, and always use full paths.

8. Additional Service – Docker Containerisation

Docker was selected as the additional server service because it is widely used throughout modern cloud environments.

Installation:

bash

```
sudo apt install docker.io -y
```

```
sudo systemctl start docker
```

```
sudo usermod -aG docker $USER # Re-login required
```

Testing and Deployment:

bash

```
docker run hello-world
```

```
docker run -d -p 8080:80 --name my-nginx nginx
```

```
docker ps
```

The Nginx container was successfully deployed and accessed through a web browser at

<http://44.195.61.45:8080>

Screenshot 22

```
ubuntu@ip-172-31-6-249:~$ sudo docker ps
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
CONTAINER ID   IMAGE      COMMAND                  CREATED    STATUS    PORTS                               NAMES
5e05d1a940f7   nginx     "/docker-entrypoint..." 39 minutes ago    Up 39 minutes    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp    my-nginx
ubuntu@ip-172-31-6-249:~$ docker --version
```

Screenshot 23

```
ubuntu@ip-172-31-6-249:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
58dee6a49ef1: Pull complete
c3bdf82c34d1: Download complete
Digest: sha256:0e760fd9bc48ba8041e7c6db999bb40bfca508b4be580ac75d32c4e29d202ce1
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (arm64v8)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

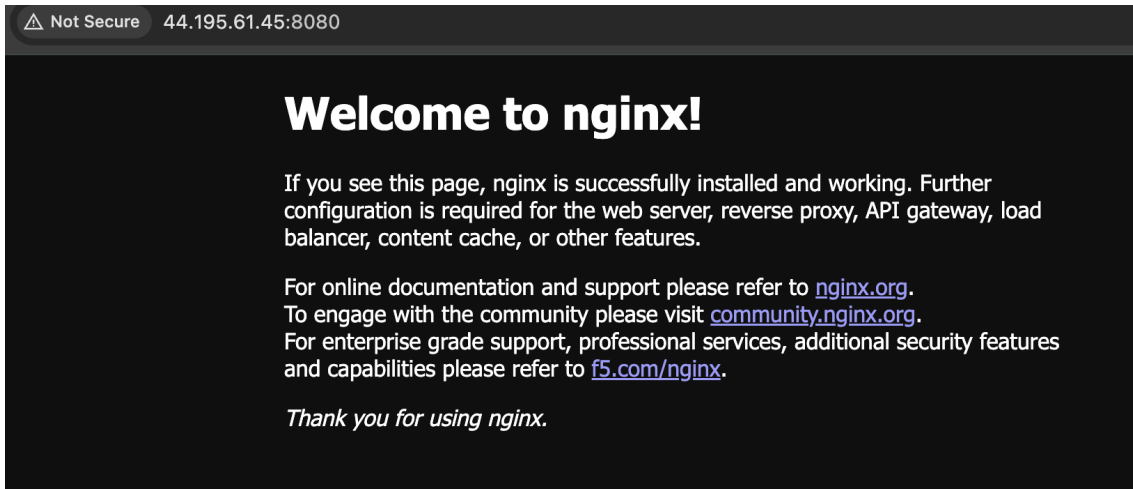
Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

Screenshot 24

```
ubuntu@ip-172-31-6-249:~$ sudo docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED    STATUS    PORTS                               NAMES
5e05d1a940f7   nginx     "/docker-entrypoint..." About a minute ago    Up About a minute    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp    my-nginx
```

Screenshot 25



Reflection: Docker was completely new to me. I installed it, ran `docker run hello-world`, and saw the confirmation message. Then I deployed an Nginx container with `docker run -d -p 8080:80 --name my-nginx nginx`. In seconds, I had a web server running. What surprised me was that I didn't need to install Nginx or configure anything — it just worked. Later I learned that Docker images include everything the app needs. This is why containers are so popular in DevOps. I also noticed that everything worked on my M1 Mac and on AWS Graviton because both are ARM64. That was a nice bonus.

9. Problems Encountered and Solutions

Problem	Solution
Installed x86_64 Ubuntu on M1 Mac	Re-downloaded ARM64 version of Ubuntu
UTM network connectivity issues	Switched from "None" to "Shared Network" mode
SSH connection to AWS failed	Configured Security Group to allow port 22 from My IP
Certbot domain validation failed	Waited 30-60 minutes for DNS propagation
Cron job did not execute	Used absolute file paths (e.g., /home/ubuntu/script.sh)

Docker permission denied	Added user to docker group: <code>sudo usermod -aG docker \$USER</code>
--------------------------	---

Reflection: Troubleshooting became one of the most valuable skills developed throughout the semester. Many issues could not be solved immediately and required systematic investigation. These experiences improved my confidence when diagnosing and resolving technical problems.

10. Industry Relevance

One thing I found valuable about ICT171 was that many of the activities reflected real tasks performed in the IT industry. Throughout the semester, I worked with Linux servers, cloud infrastructure, automation tools, security configurations, and container technologies that are commonly used in professional environments. The skills I developed can be applied to a variety of technical roles.

Role	Relevant Skills
System Administrator	Linux CLI, service management (systemctl), permissions (chmod/chown), firewall (UFW)
Cloud Engineer	AWS EC2, Security Groups, ARM64 Graviton instances, cost management
DevOps Engineer	Docker containers, Bash scripting, cron automation, infrastructure as code
Cybersecurity Analyst	Log analysis (grep/awk), TLS/SSL, access control, firewall configuration

I also found it interesting that both my MacBook M1 and AWS Graviton instances use ARM architecture. At first, this caused some compatibility challenges when installing Ubuntu because I accidentally downloaded the wrong version. After resolving the issue, I gained a better understanding of ARM64 systems and why they are becoming increasingly common in cloud environments. Learning on Apple Silicon gave me practical experience with a technology that is now widely adopted in modern data centres.

Overall, this unit helped me understand how the different technologies covered in class work together in real-world environments. Rather than learning each topic in isolation, I was able to see how Linux, cloud computing, networking, security, automation, and containerisation combine to support modern IT operations.

11. Final Reflection

ICT171 significantly expanded my practical IT knowledge and confidence. At the beginning of the semester, my Linux experience was limited and cloud technologies appeared complex. Through hands-on activities, I learned how to:

- Deploy cloud servers on AWS EC2
- Manage Linux systems using command-line tools
- Secure websites using TLS certificates (Let's Encrypt)
- Automate administrative tasks using Bash scripts and cron
- Deploy applications using Docker containers

Key improvements:

Skill	Before ICT171	After ICT171
Linux CLI	Minimal	Confident with 50+ commands
Cloud computing	Abstract concept	Hands-on AWS EC2 deployment
Scripting	Never written a script	Automated backup and monitoring
Docker	Only heard the name	Can deploy and manage containers
Security	"Green lock" only	Understand SSL, firewalls, permissions

I came into ICT171 knowing very little about Linux or cloud computing. At the beginning of the semester, many of the commands and concepts seemed unfamiliar, and I was not confident using the Linux command line. By the end of the unit, I was able to do things I never thought I could, including launching AWS servers, connecting through SSH, installing and configuring Apache, securing websites with HTTPS, writing Bash scripts, automating tasks with cron, and deploying Docker containers.

One of the biggest lessons I learned was that technical problems rarely have instant solutions. Throughout the semester, I encountered several challenges, including downloading the wrong Ubuntu version for my MacBook M1, troubleshooting UTM networking issues, configuring AWS security groups, fixing cron jobs that would not execute, and resolving Docker permission errors. Although these situations were frustrating at the time, they forced me to investigate problems systematically and improved my troubleshooting skills significantly.

I also gained a much better understanding of how different technologies work together in modern IT environments. Before this unit, topics such as Linux administration, cloud computing, networking,

security, automation, and containerisation seemed like separate concepts. Through the practical labs, I learned how these technologies interact to support real-world systems and services.

If I were to complete this unit again, I would commit my work to GitHub more frequently throughout the semester instead of leaving most uploads until the end. I would also spend more time exploring advanced Docker topics such as networking, volumes, and multi-container deployments. These areas seem increasingly important in cloud and DevOps environments.

For other MacBook M1 users, I would strongly recommend downloading the ARM64 version of Ubuntu and using UTM rather than VirtualBox. I would also recommend experimenting with AWS Graviton instances because they use the same ARM architecture and provide valuable hands-on experience with modern cloud infrastructure.

Overall, ICT171 provided me with a strong foundation in Linux administration, cloud computing, automation, security, and containerisation. More importantly, it gave me confidence to learn independently, troubleshoot technical problems, and work with technologies commonly used in the IT industry. I feel much better prepared for future studies and career paths related to System Administration, Cloud Engineering, DevOps, and Cybersecurity.

Looking back, the most valuable skill I gained is not any single command or tool, but the ability to troubleshoot problems I've never seen before. Early in the semester, an error message would stop me completely. Now I know where to look and what to search for.

12. AI Tools Used

During this unit, I used AI tools to augment — not replace — my learning:

Tool	Purpose
ChatGPT	Explaining error messages and suggesting solutions
GitHub Copilot	Script syntax suggestions and completion
Grammarly	Grammar checking and writing clarity

Statement: For example, when I received a 'permission denied' error while running Docker, I asked ChatGPT what the error meant and it suggested adding my user to the docker group. I then ran `sudo usermod -aG docker $USER` myself and the problem was solved.

All commands were executed by me, all scripts were written and tested by me, and all screenshots are from my own work. AI tools were used only for explanation, syntax suggestions, and grammar checking. All AI-generated suggestions were verified before implementation.

Screenshot Index

Figure	Lab Section	Description
Screenshot 1	2.1	GitHub repository page showing ICT171-Lab-Journal repository
Screenshot 2	2.2	UTM main interface showing Ubuntu ARM64 virtual machine
Screenshot 3	2.2	Terminal output of <code>uname -m</code> showing <code>aarch64</code>
Screenshot 4	2.3	Terminal output of <code>ls -la</code> command
Screenshot 5	3.1	<code>sudo systemctl status apache2</code> showing active (running)
Screenshot 6	3.1	<code>sudo ufw status</code> showing allowed ports
Screenshot 7	3.2	<code>ls -l /home/shared</code> showing file permissions
Screenshot 8	3.3	<code>find</code> and <code>grep</code> command search results
Screenshot 9	4.1	TCO comparison table (Excel or screenshot)
Screenshot 10	4.2	AWS EC2 console showing running t4g.micro instance
Screenshot 11	4.2	Terminal showing successful SSH connection to EC2

Screenshot 12	4.2	Browser showing Apache welcome page (HTTP)
Screenshot 13	5.1	backup_script.sh script code
Screenshot 14	5.1	Script execution output
Screenshot 15	5.2	grep with regex output for Apache log analysis
Screenshot 16	6.1	nslookup reverse DNS lookup for AWS EC2 instance
Screenshot 17	6.1	dig reverse DNS query for AWS EC2 instance
Screenshot 18	6.2	certbot --version output
Screenshot 19	6.2	certbot renew --dry-run verification
Screenshot 20	7.0	crontab -e content screenshot
Screenshot 21	7.0	List of generated backup ZIP files
Screenshot 22	8.0	docker --version output
Screenshot 23	8.0	docker run hello-world successful output
Screenshot 24	8.0	docker ps showing running nginx container
Screenshot 25	8.0	Browser showing Nginx welcome page (port 8080)

References

Amazon Web Services. (2024). Amazon EC2 documentation – Virtual servers in the cloud. Retrieved from <https://docs.aws.amazon.com/ec2/>

Amazon Web Services. (2024). AWS Graviton processors – Arm-based compute on AWS. Retrieved from <https://aws.amazon.com/ec2/graviton/>

Canonical. (2024). Ubuntu server documentation – ARM64 support. Retrieved from <https://ubuntu.com/server/docs>

Docker Inc. (2024). Docker documentation – Get started with containers. Retrieved from <https://docs.docker.com/>

Docker Inc. (2024). *Docker on ARM64 – Multi-platform images*. Retrieved from <https://docs.docker.com/build/building/multi-platform/>

Electronic Frontier Foundation. (2024). Certbot documentation – Free SSL/TLS certificates. Retrieved from <https://certbot.eff.org/>

Internet Security Research Group. (2024). Let's Encrypt documentation – Free SSL/TLS certificates. Retrieved from <https://letsencrypt.org/>

Namecheap. (2024). DNS management documentation. Retrieved from <https://www.namecheap.com/>

UTM Team. (2024). UTM documentation – Virtualisation for Apple Silicon. Retrieved from <https://utm.app/>

WordPress. (2024). [WordPress.com](https://wordpress.com) pricing and plans. Retrieved from <https://wordpress.com/pricing/>